# 02.Introduction aux frameworks PHP

19 octobre 2026

## Développement web il3

### Introduction aux frameworks PHP

HE-Arc 2016-26 DGR et YBL

---

## Framework [1]

- Fonctionnalités similaires pour de nombreuses applis
- Composants de haut-niveau réutilisables (faible couplage)
- Règles de codage et d'architecture
- Code sûr et efficace
- Facilite les tests et la gestion de projets complexes
- Utilisation de Design Patterns dès que possible
- Comportement par défaut
- Extensible
- Principe d'inversion de contrôle

Différences entre framework et library sur Stack Overflow [2] ou artima developper [3].

1. http://en.wikipedia.org/wiki/Software_framework
2. http://stackoverflow.com/questions/148747/what-is-the-difference-between-a-framework-and-a-library
3. http://www.artima.com/forums/flat.jsp?forum=106&thread=152104

## Design Patterns et webdev

- Inversion de contrôle (IoC [4])
- Model View Controller
  - M : Accès aux données, logique métier
  - V : Templates des pages à générer
  - C : Orchestration, transfert des infos
- Front Controller
  - Traitement et dispatch des requêtes grâce aux **routes**
  - (bootstrap, ré-écriture des URL, ...)
- Object Relational Mapping [5]
  - Active Record, Table Data Gateway, Data Mapper, ...
- UI Patterns [6]

## MVC for webdev

## Conventions

- Nommage
  - Classes
  - Base de données
  - Fichiers et dossiers
- ROUTES : `http://app.host.tld/controller/action[/key/val]`
- Arborescence :
  - Imposée ou libre selon frameworks
  - Pas de code (minimum) sous la racine web
- Conventions obligatoires ou non, mais RECOMMANDEES dans tous les cas

## Bonnes pratiques

- Heavy Model, Light Controller
- Don't Repeat Yourself
- You Ain't Gonna Need It
- Convention Over Configuration
- Keep It Simple and Stupid

4. http://martinfowler.com/bliki/InversionOfControl.html
5. https://web.archive.org/web/20160316065751/http://blog.mazenod.fr/2010/01/design-pattern-mvc-zoom-sur-la-couche-modele-dal-dao-orm-crud/
6. http://ui-patterns.com/

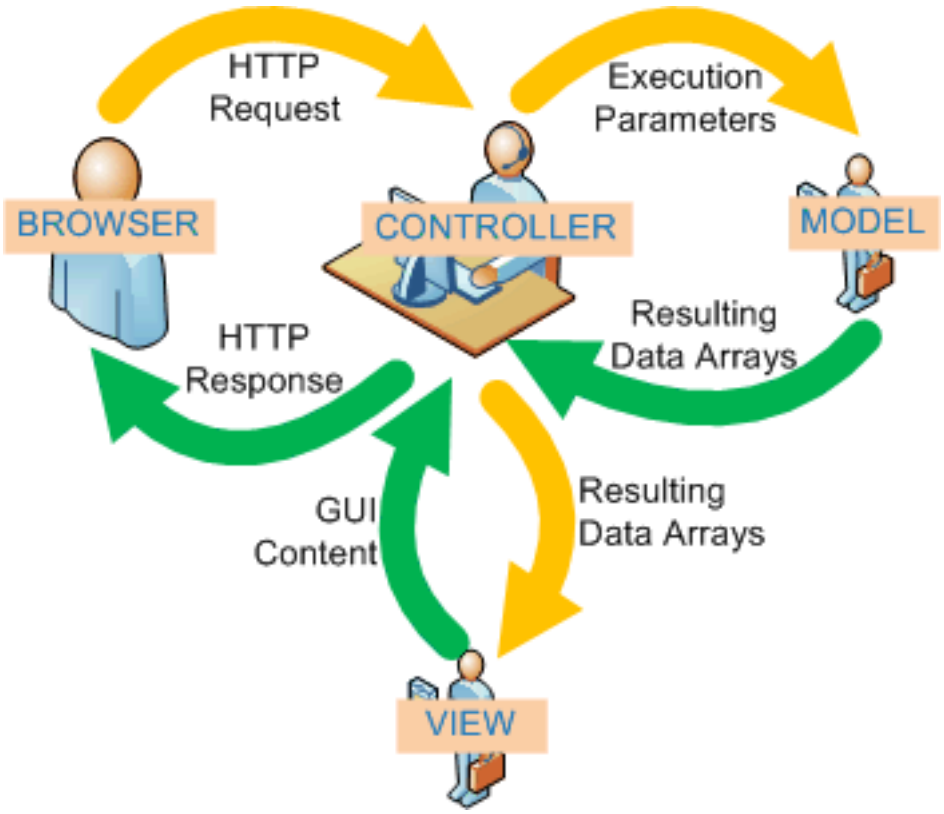

HTTP
Request
Execution
Parameters
BROWSER
CONTROLLER
MODEL
HTTP
Response
Resulting
Data Arrays
GUI
Content
Resulting
Data Arrays
VIEW

Figure 1 – MVC

— 12 factor app [7] - fr [8]

## Pretty ( | smart | clean | formatted) URL

— Les URL doivent être explicites :
  — Manipulées par l'utilisateur
  — Utilisées pour le référencement
— Cohérence avec l'implémentation MVC :

```
http://app.host.tld/controller/action[/key/val]
```

— Le routage (routing)
  — Le Front Controller recoit toutes les requêtes (URL rewriting)
  — Il les dispatche vers les contrôleurs

## Smart URL & SEO

## Autres Services

— Migrations : Evolutions de la strucutre de la BDD
— Tests
— Génération, validation et traitement de formulaires
— Authenfication, Sessions, Permissions, Roles, ACL
— Pagination
— I18n
— Génération de code
— Mail
— Connecteurs aux webservices
— Captchas
— Loggers
— ...

## Exemple d'architecture : Laravel

Ce schéma est clair mais pas tout à fait juste : dans Laravel, le contrôleur récupère la page générée à partir de la vue, et c'est lui qui renvoie le HTML (objet Response) au client.

7. https://12factor.net/
8. https://12factor.net/fr/

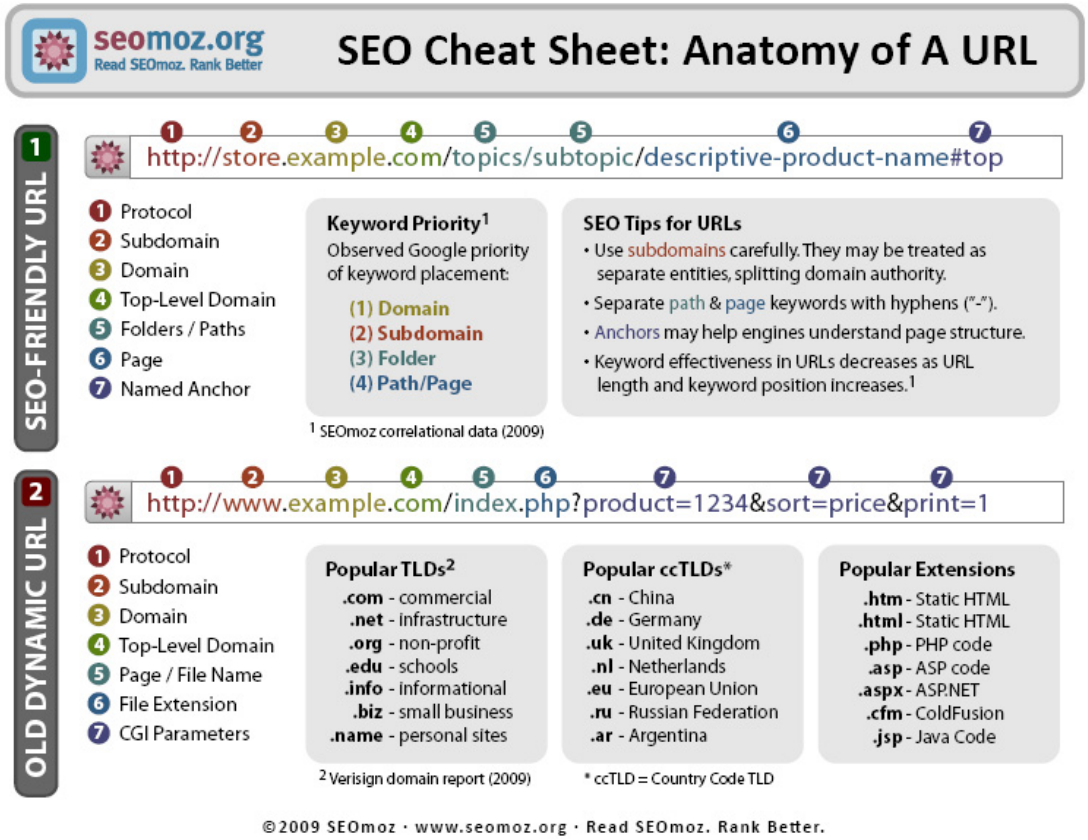



FIGURE 2 – SEO

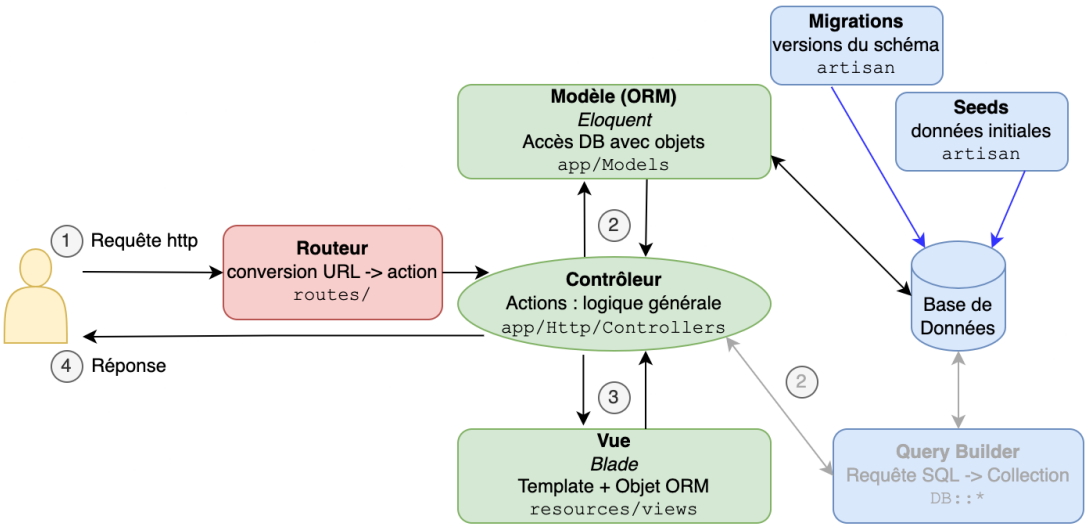



FIGURE 3 – Archi

## Performance

- Un framework web est lent :
  - Rendu d'une page nécéssite de traverser tout le code
  - Pour chaque requête toute l'appli est chargée
  - Plus de code qu'une appli standalone
  - Plus de requêtes
- Solutions
  - Cache de pages, d'opcode
  - Jointures ORM, vues, procédures stockées
  - Outils d'optimisation : YSlow, page speed, mytop

## Frameworks PHP

- Lesquels connaissez-vous ?
- Lesquels avez-vous utilisé ?
- Pourquoi y en a-t-il tant ?

L'explication donnée par Joe Gregorio pour le langage Python [9] est : « parce que c'est facile. »

Dans les faits, cela montre également une maturité de la plateforme.

---

> *There are people who actually like programming. I don't understand why they like programming.* Rasmus Lerdorf 💬 [10]

---

- PHP-FI *Forms Interpreter*
- PHP 3, réécrit en C++
- PHP 4 *Zend Engine*, fausse POO
- PHP 5, vraie POO, PDO, JSON, …
- ~~PHP 6, Unicode~~ 💩, 🎃, 🐧
- PHP 7, que du rêve !
- PHP 8, JIT compilation, … [11]

9. http://bitworking.org/news/Why_so_many_Python_web_frameworks
10. https://en.wikiquote.org/wiki/Rasmus_Lerdorf
11. https://www.php.net/releases/8.5/fr.php

Il y a plus de vingt ans, Rasmus Lerdorf bricola un outil pour savoir qui consultait son CV.

Zend, c'est à dire *ZEev* et *aNDi*, ont réécrit PHP et qui allait devenir PHP 3 le précurseur du langage de prédilection pour créer sur le web.

PHP a évolué depuis pour devenir ce qu'il est aujourd'hui. Sa popularité est liée au fait qu'il est simple à mettre en œuvre, gratuit **et** libre. Tout un tas de modules est fourni avec pour faire de l'imagerie, des bases de données, du XML, etc.

Et plus encore sur la page History of PHP [12] et Wikipedia : PHP [13].

Les différentes moutures de PHP 7 offrent ceci, entre autres.

— PHP 7, performances
— PHP 7.1, `void`
— PHP 7.2, sodium

---

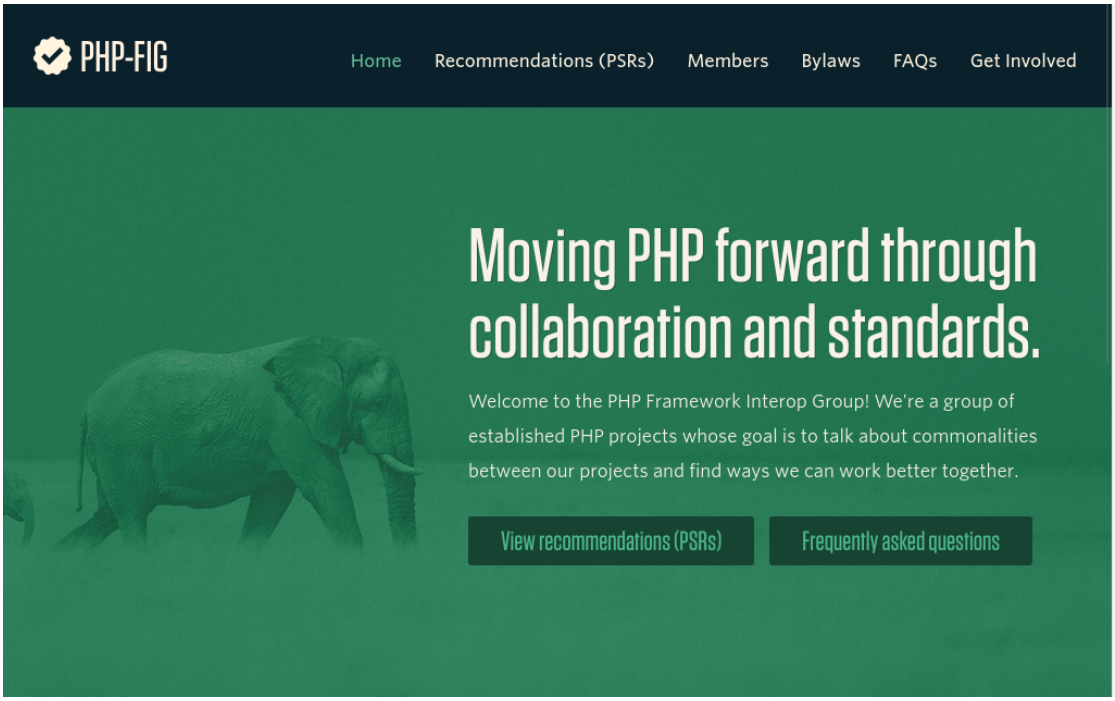



Figure 4 – PHP Framework Interop Group

12. http://php.net/manual/en/history.php.php
13. https://en.wikipedia.org/wiki/PHP

L'évolution de PHP a fait que les usagers du langage, créateur de *frameworks*, d'outils (comme *Composer*[14]), ont senti le besoin d'émettre des recommendations afin d'aller vers un plus interopérable.

Durant ce cours, nous allons vous embêter avec PSR-1, PSR-2 et PSR-4.

---

## Quiz

Qui est qui ?

---



FIGURE 5 – source [15]

oOops, ceci n'a rien à voir avec le cours.

---

14. http://getcomposer.org/



(1)

Donc, ce ne sont pas Gandalf (sans sa barbe) et Saruman mais bien Sir Tim Berners-Lee et Vinton Cerf, responsables du (World Wide) Web et de l'Internet.

---

## Qu'est-ce qu'Internet [16] ?

— un réseau IP

---

## Qu'est-ce que le World Wide Web [17] ?

— **URI/URL**, des identifiants uniques
— **HTML**, un langage de publication
— **HTTP**, un protocole d'échange de texte (ou *HyperText*)

---

16. https://www.youtube.com/watch?v=iDbyYGrswtg
17. https://line-mode.cern.ch/

## Préparatifs

https://github.com/HE-Arc/php-intro-framework/ [18]

```
$ sudo systemctl start httpd
$ cd /var/www/html
$ git clone \
> https://github.com/\
> HE-Arc/php-intro-framework

$ cd php-intro-framework
$ open http://localhost/php-intro-framework
```

Les exemples suivant travaillent sur le code disponible dans le dépôt HE-Arc/php-intro-framework [19].

---

```
$ curl -v "http://he-arc.ch/?id=25"
> GET /?id=25 HTTP/1.1
> Host: he-arc.ch
>
< HTTP/1.1 200 OK
< Content-Type: text/html; charset=utf-8
<

<!DOCTYPE html>
<title>HE-Arc</title>
<p>Hello
```

HTTP est un protocole texte plutôt simple, jugez plutôt :

Ce que nous voyons est une connexion TCP/IP au serveur `he-arc.ch`. Une fois la connexion établie, il envoie en texte ASCII les entêtes HTTP puis deux retours à la ligne (ce qui correspond à une ligne vide). La requête HTTP commencent toujours par la demande, ici `GET /index.php?page=equipe&id=25 HTTP/1.1` puis les entêtes, ici : `Host: www.he-arc.ch`. La réponse du serveur est du même type, le code de réponse (`HTTP/1.1 200 OK`), les entêtes, une ligne vide puis le contenu.

La demande et les entêtes sont en US-ASCII mais le corps peut être encodé autrement, ici c'est dit dans l'entête `Content-Type: text/html; charset=utf-8`.

18. https://github.com/HE-Arc/php-intro-framework
19. https://github.com/HE-Arc/php-intro-framework

## Fait #1

PHP parle HTTP.

Le fichier `index.php` est le code PHP le plus simple qui soit. Simple au sens du niveau de compréhension de PHP et d'une forme de complexité.

```php
<?php // 00-base

// Lecture de la query string `page=<XX>&id=<YY>`.
$page = $_GET["page"] ?? null;
$id = (int) ($_GET["id"] ?? 0);

// Connexion à la base de données.
$db = new PDO("sqlite:../users.db");

// Page HTML
?>
<!DOCTYPE html>
<meta charset=utf-8>
<title>HE-Arc</title>
<?php
// Contenu
if ("equipe" === $page):
    $query = $db->query("SELECT * FROM `personnes` WHERE `id` = :id;");
    $query->execute(compact('id'));

    $personne = $query->fetch(PDO::FETCH_OBJ);
?>
    <p><a href="<?php echo $_SERVER["PHP_SELF"] ?>">retour</a>
    <h1>Équipe</h1>
    <h2>
        <?php echo $personne->prenom ?>
        <?php echo $personne->nom ?>
    </h2>
    <p>
        <img src="//www.gravatar.com/avatar/<?php
            echo md5(strtolower($personne->email));
        ?>" alt="avatar">
<?php
else:
?>
```

```
    <h1>Accueil</h1>
    <ul>
        <li><a href="?page=equipe&amp;id=1">Yoan Blanc</a>
        <li><a href="?page=equipe&amp;id=2">Yoan Blanc</a>
    </ul>
<?php
endif
```

---

### Fait #2

PHP **est** un langage de template.

Pour preuve, il faut ouvrir une balise `<?php` pour commencer la partie code.

Avec la pratique, on a réalisé que mélanger la logique métier et celle d'affichage n'est pas optimal car difficile à lire et maintenir.

---

## Séparation métier/affichage

```php
<?php // 01-includes/index.php

// ...

include "templates/entete.html";

if ("equipe" === $_GET["page"]) {
    // SELECT FROM u WHERE id=$_GET["id"]
    // ...
    include "templates/equipe.html";
} else {
    // ...
    include "templates/accueil.html";
}
```

---




Quel est le problème avec cette solution ?

(Source de l'image [20])

---

## Sécurité des templates

— *Principle of Least Privilege* ( polp [21] )
— Intégration faite par un graphiste, société externe

Dans ce le cas présent rien ne nous empêche de mettre de la logique métier dans nos fichiers de *template*, car ils sont faits de PHP eux aussi.

---

20. https://raw.githubusercontent.com/cyrilmanuel/picbot/e6ff24a8bfd7ee9f0514a4fd8f49b1255ef26178/picbot/Images/meme10.jpg
21. https://en.wikipedia.org/wiki/Principle_of_least_privilege

```
{# 02-twig/templates/collaborateur.html #}
{%- extends "base.html" -%}

{% block corps -%}
<p><a href="?">retour</a>
<h1>Équipe</h1>
<h2>
  {{- personne.prenom -}}
  {{ personne.nom -}}
</h2>
<p><img
  src="//www.gravatar.com/avatar/
  {{- personne.email | strtolower | md5 }}"
  alt="avatar">
{% endblock -%}
```

La page est réalisée avec Twig [22] <2.0. À partir de la version 2.0, il faut utiliser un *autoloader* externe, comme celui de composer (voir ci-dessous).

Le code est un poil plus propre du côté de nos *templates* qui ne peuvent plus exécuter de PHP sauf ce qu'on leur autorise, ici `md5` et `strtolower`. Voir `02-twig/index.php` [23].

```
<?php // 02-twig

require_once 'Twig/lib/Twig/Autoloader.php';
Twig_Autoloader::register();

// ...

// Configuration de Twig
$loader = new Twig_Loader_FileSystem("templates");
$twig = new Twig_Environment($loader);

// Ajout des filtres md5 et strtolower qui sont les fonctions PHP du même nom.
$twig->addFilter(new Twig_SimpleFilter('strtolower', 'strtolower'));
$twig->addFilter(new Twig_SimpleFilter('md5', 'md5'));

// variable globale
$titre = "HE-Arc";

// Contenu
if ("equipe" === $page) {
```

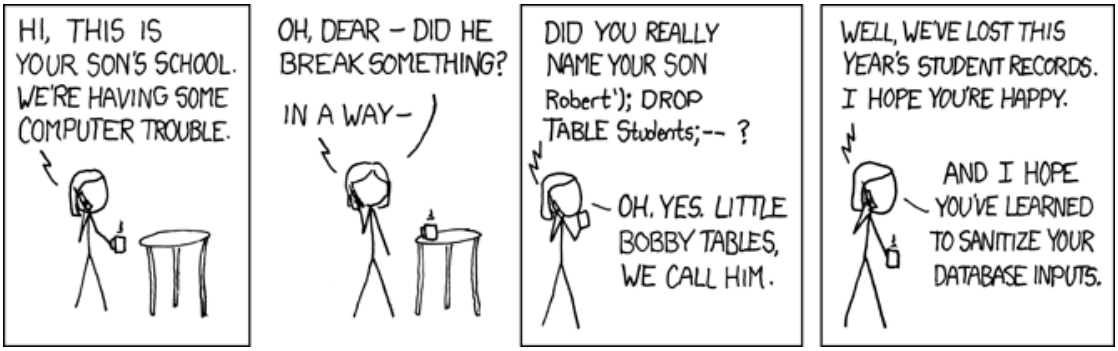
22. http://twig.sensiolabs.org/
23. 02-twig/index.php

```
    // ...
    $personne = // ...

    echo $twig->render("equipe.html", compact("titre", "personne"));
} else {
    $personnes = // ...

    echo $twig->render("accueil.html", compact("titre", "personnes"));
}
```

---




Problème d'injection SQL.

Effectuer des requêtes MySQL à la main ou devoir connaitre tous les champs crée beaucoup de redondance et de failles de sécurité potentielles.

Une solution est d'ajouter une couche d'abstraction qui va cacher la structure réelle de notre base de données et offrir une interface orientée objet. Un *Object-Relational Mapping* ou ORM(3) dans le jargon.

---

```
<?php
// Ne dites plus
$query = $db->query(
  "SELECT * FROM `personnes` ".
  "WHERE `id` = :id;"
);
$query->execute(compact('id'));
$personne = $query->fetch(PDO::FETCH_OBJ);
```

```php
// Mais dites plutôt

//  RedBean
$personne = R::load('personnes', $id);
// ou Doctrine
$personne = $om->find('Personne', $id);
```

---

## *Object-Relational Mapping*

- RedBean [24]
- Doctrine [25] (ORM, ODM)
- Eloquent ORM [26]
- etc. [27]

Une bibliothèque qui va créer ce lien entre les mondes objet et relationnel ou document (généralement MongoDB). Il en existe toute une foule.

---

```php
<?php // 03-redbean/index.php
require 'RedBean/rb.php';
R::setup("sqlite:../users.db");
// ...
if ("equipe" === $page) {
    $personne = R::load("personnes", $id);
    echo $twig->render(
        "equipe.html",
        compact("titre", "personne")
    );
} else {
    $personnes = R::find("personnes");
    echo $twig->render(
        "accueil.html",
        compact("titre", "personnes")
    );
}
```

24. http://www.redbeanphp.com/
25. http://www.doctrine-project.org/
26. http://laravel.com/docs/master/eloquent
27. https://en.wikipedia.org/wiki/List_of_object-relational_mapping_software#PHP

---

## URI as UI

Pensez à Wikipedia.

Les adresses des pages font partie de l'expérience utilisateur. Un utilisateur doit être capable d'imaginer le contenu de la page en lisant l'URI. Certainement, ce que vous faites avant de cliquer sur un lien.

---

### Comment humaniser ?

```
/index.php?page=equipe&id=42
```

La personne avec l'identifiant `42` aura également un *slug* unique créé à partir de son nom, ici `jean-bon`.

La solution à notre problème est de demander au serveur web de réécrire les URL pour nous.

---

### Réécriture d'URL

```
# 04-routes/.htaccess

# mod_rewrite
RewriteEngine on
RewriteBase /php-intro-framework/04-routes/

RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php/$1 [L,QSA]
```

Apache le fait via `mod_rewrite` [28] et Nginx `try_files` [29].

---

28. https://httpd.apache.org/docs/current/mod/mod_rewrite.html
29. http://nginx.org/en/docs/http/ngx_http_core_module.html#try_files

```php
// 04-routes/index.php

$uri = $_SERVER['REQUEST_URI'],
$matches = [];

preg_match(
    "#^/(?P<page>[^/]+)/(?P<slug>[^/]+)/?#",
    $uri,
    $matches
) or die('Arrrrrgh');

echo call_user_func_array(
    $matches['page'],
    [$matches['slug']]
);
```

Le code complet va nettoyer l'URI et définir les fonction correspondant aux pages possibles.

---

## *Routing*

Lien entre les adresses (URI) et des actions dans le code.

a.k.a. the *Front Controller*.

En pratique, les actions ne sont pas des fonctions mises à plat mais sont encapsulées dans une classe qu'on nomme un contrôleur. Faire ainsi permet de regrouper logiquement les fonctions et éviter d'utiliser d'affreux éléments tel que `global`.

---

## Modèle - Vue - Contrôleur

- Modèle : l'ORM qui s'occupe de notre base de données
- Vue : les templates qui affiche les données
- Contrôleur : une classe qui définit quoi faire en fonction des entrées utilisateur (URI, formulaire, etc.)

*MVC*(4) vient des applications bureau et ne représente pas toujours le fonctionnement dans le monde du web. Par exemple, Django, un framework Python, se décrit comme étant *Modèle - Template - Vue*(5).

Les frameworks web en PHP (ou d'autres langages) reposent majoritairement sur ce paradigme.

---

## *Composer*

Gestionnaire de paquets pour PHP : getcomposer.org [30]

Maintenir notre répertoire de *vendor* ainsi que les `require` est peu pratique. Voici qu'entre en scène Composer [31], le gestionnaire de paquet pour PHP. Packagist [32] est le dépôt en ligne de paquets public et utilisé par défaut.

---

### composer.json

```
{
    "require": {
        "twig/twig": "^2.0",
        "gabordemooij/redbean": "^4.3",
    }
}
```

Nos dépendances sont ainsi matérialisées dans le projet et peuvent être installée, ou mises à jour simplement.

En principe les numéros de version respectent le SemVer [33] (*Semantic Versioning*) et les différents signes permettent de sélection une ou plusieurs versions (voir [Version and constraints][https://getcomposer.org/doc/articles/versions.md]).

---

30. http://getcomposer.org/
31. http://getcomposer.org/
32. https://packagist.org/
33. http://semver.org/lang/fr/

```
$ composer install
```

puis

```
<?php // 05-composer/index.php

require 'vendor/autoload.php';

use RedBeanPHP\Facade as R;
```

Enfin, nous pouvons réduire le nombre de `require` et `include` à un seul, en laissant soin à l'*auto-loader* de charger le bon fichier à la demande. Tout ceci est spécifié dans PSR-4 [34]. Ainsi, les définitions de Twig sont présentes et il nous suffit d'obtenir la classe `R` depuis RedBean [35].

---

## *Front-Controller*

Utilisation de FastRoute [36] (voir 06-fastroute/index.php [37]).

```
$ composer require nikic/fast-route
```

`FastRoute` repose sur un système proche de celui que nous avons utilisé jusqu'ici. D'autres systèmes, tels que `Aura.Router` pour ne citer que lui, reposent sur la spécification PSR-7 [38]. Cette dernière décrit l'interface objet d'un message HTTP, tant au niveau de la requête que de la réponse.

Si ça ajoute, une bonne couche de complexité, l'énorme avantage offert par cette idée là est de déléguer le rendu d'une page, ni `echo`, ni `header`, Donc il est envisageable de pouvoir tester (au sens de test unitaire), notre *FrontController*.

D'autre part, le `call_user_func_array` d'avant n'était pas très solide,

---

34. http://www.php-fig.org/psr/psr-4/
35. http://www.redbeanphp.com/
36. https://github.com/nikic/FastRoute
37. https://github.com/HE-Arc/php-intro-framework/blob/master/06-fastroute/index.php
38. http://www.php-fig.org/psr/psr-7/

```php
<?php // 06-fastroute/index.php
// ...
use function FastRoute\simpleDispatcher;
use FastRouter\Dispatcher;

$dispatcher = simpleDispatcher(function($r)
{
    $r->addRoute('GET', '/', 'accueil');

    $r->addRoute(
        'GET',
        '/equipe/{slug}',
        'equipe'
    );
});
```

---

```php
<?php // 06-fastroute/index.php (suite)

$httpMethod = $_SERVER["REQUEST_METHOD"];
$uri = $_SERVER["REQUEST_URI"];

// nettoyage de $uri
// - prefix
// - query string
// - caractères spéciaux (e.g. %20)

$routeInfo = $dispatcher->dispatch(
    $httpMethod,
    $uri
);
```

---

```php
<?php // 06-fastroute (suite)

switch($routeInfo[0]) {
    case Dispatcher::NOT_FOUND:
    case Dispatcher::METHOD_NOT_ALLOWED:
```

```
        /* ... */break;
    case Dispatcher::FOUND:
        try {
            echo call_user_func_array(
                $routeInfo[1],
                $routeInfo[2]
            );
        } catch (Exception $e){
            echo server_error($e);
        }
        break;
}
```

---

## *Framework PHP*

Une collection de bibliothèques avec un peu de glue.

Un framework web vous propose une structure de base pour construire selon une méthode jugée bonne par ses concepteurs. Il est possible de remplacer un composant par un autre, par le sien. Et même de créer sa *glue* ou même ses outils propres.

---

### Liens avec Laravel

- Modèle MVC
- Templates utilisant *blade*.
- ORM nommé *Eloquent*.
- *Front-Controller* (`Illuminate\Routing`)
- Bibliothèques … (`Illuminate\*`)
- Composer [39]

Je vous invite à aller lire le code généré pour vous par Laravel. Vous allez retrouver ces éléments. Symfony, CakePHP, etc. auront les mêmes idées.

---

39. http://getcomposer.org/

### Exercice

— Refaites les différentes étapes à partir de `00-base`.
— Tel quel ou en utilisant d'autres bibliothèques : Smarty [40], Doctrine [41], Aura.Router [42]

---

## Fin

Questions ?

## Sources


1. W3C. W3C 20 Anniversary Symposium. [en ligne]. 2014. [Consulté le 7 février 2017]. Disponible à l'adresse : https://www.w3.org/20/Overview.html

2. MUNROE, Randall. Exploits of a mom. [en ligne]. 2007. [Consulté le 7 février 2017]. Disponible à l'adresse : https://xkcd.com/327/

3. WIKIPEDIA. *Mapping objet-relationnel* [en ligne]. [Consulté le 7 février 2017]. Disponible à l'adresse : https://fr.wikipedia.org/wiki/Mapping_objet-relationnel

4. WIKIPEDIA. Modèle-Vue-Contrôleur. [en ligne]. [Consulté le 7 février 2017]. Disponible à l'adresse : https://fr.wikipedia.org/wiki/Modèle-vue-contrôleur [43]

5. DJANGO PROJECT. Django appears to be a MVC framework, but you call the Controller the "view", and the View the "template". How come you don't use the standard names ? *FAQ : General* [en ligne]. [Consulté le 7 février 2017]. Disponible à l'adresse : https://docs.djangoproject.com/en/1.11/faq/general/#django-appears-to-be-a-mvc-framework-but-you-call-the-controller-the-view-and-the-view-the-template-how-come-you-don-t-use-the-standard-names


40. https://github.com/smarty-php/smarty
41. https://www.doctrine-project.org/projects/doctrine-orm/en/current/tutorials/getting-started.html
42. https://github.com/auraphp/Aura.Router